

Efficient Single Image Super-Resolution

Using Frequency-Inspired Optimization in Transformer

Tushar Moitra

Indian Institute of Technology Dharwad

April 24, 2025

1. Introduction to Super-Resolution
2. Abstract and Other Works
3. CRAFT Components
4. Novel Improvements on CRAFT
 - 4.1 Adaptive SRWAB
 - 4.2 Edge Aware CNN Modules
 - 4.3 Refined fusion of features (HFB)
5. Frequency-Guided PTQ Strategy
6. Appendix

Introduction to Super-Resolution

- **Super-Resolution (SR)** aims to reconstruct a high-resolution (HR) image from a low-resolution (LR) input.
- Transformer-based approaches have shown promising results but struggle with high-frequency details.
- This paper introduces the **CRAFT model** to combine CNN and Transformer benefits.

- Transformers have been useful to extract long range dependencies.
- However, it is found that transformer structures are better at capturing low-frequency information, and have limited capacity in constructing high-frequency representations when compared to their convolutional counterparts
- The paper proposes a **CRAFT** transformer which is claimed to carry advantages of transformer and convolutional structures.
- Has three components namely **HFERB** for high-freq enhancement, **SRWAB** for global information and **HFB** for refining representation.
- Introduction of frequency-guided post-training quantization (PTQ) method aimed at enhancing CRAFT's efficiency, which also incorporate adaptive dual clipping and boundary refinement.
- PTQ strategy also extended to function as a general quantization method for transformer-based SISR techniques

- Introduces the cross-refinement adaptive feature modulation transformer (CRAFT), an innovative model that effectively harnesses the strengths of both CNNs and transformers.
- Use of a building block known as CRFB - Cross-Refinement Fusion Blocks.
- Multiple CRFBs form a RCRFG - Residual Cross-Refinement Fusion Group.
- These CRFBs integrate high-frequency information with long-range dependencies.

Residual Cross-Refinement Fusion Group (RCRFG)

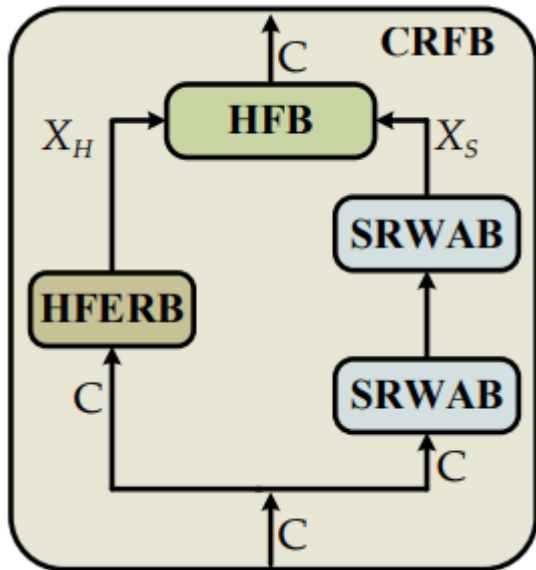
- RCRFG is the core module of CRAFT, responsible for feature enhancement.
- It consists of multiple Cross-Refinement Fusion Blocks (CRFBs).

Block

Consists of

- High-Frequency Enhancement Residual Block (HFERB)
- Shift Rectangle Window Attention Block (SRWAB)
- Hybrid Fusion Block (HFB)

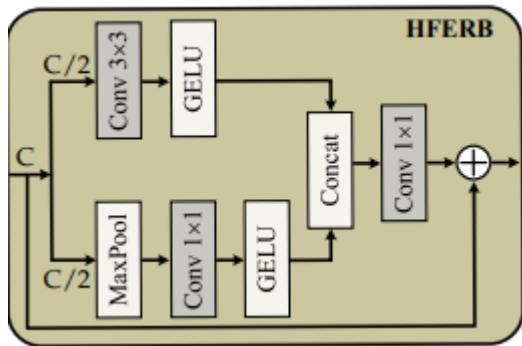
CRFB



HFERB: High-Frequency Enhancement Residual Block

- To handle and construct high frequency data, an image feature representation is used for the HFERB.
- It consists of multiple Convolutional and Max-Pooling blocks.
- It comprises the local feature extraction (LFE) branch and the high-frequency enhancement (HFE) branch.
- LFE using 3x3 Convolution block and GELU.
- HFE using max pool, 1x1 convolution and GELU.
- Concatination and use of Skip Connection for stability and gradient.

HFERB



SRWAB: Shift Rectangle Window Attention Block

- To handle long range dependencies.
- Input divided into rectangles of $H \times W \frac{1}{rh \times rw}$
- It comprises query, key and value

$$Q_i = X_i W, \quad K_i = X_i W_K, \quad V_i = X_i W_V$$

for each i th rectangle

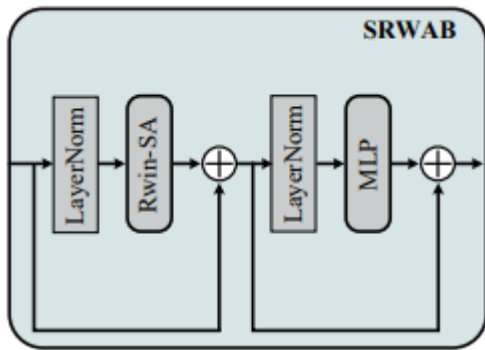
- Self Attention as:

$$\text{Attention}(Q_i, K_i, V_i) = \text{Softmax} \left(\frac{Q_i K_i^T}{\sqrt{d}} + B \right) V_i$$

where B is the dynamic relative position encoding.

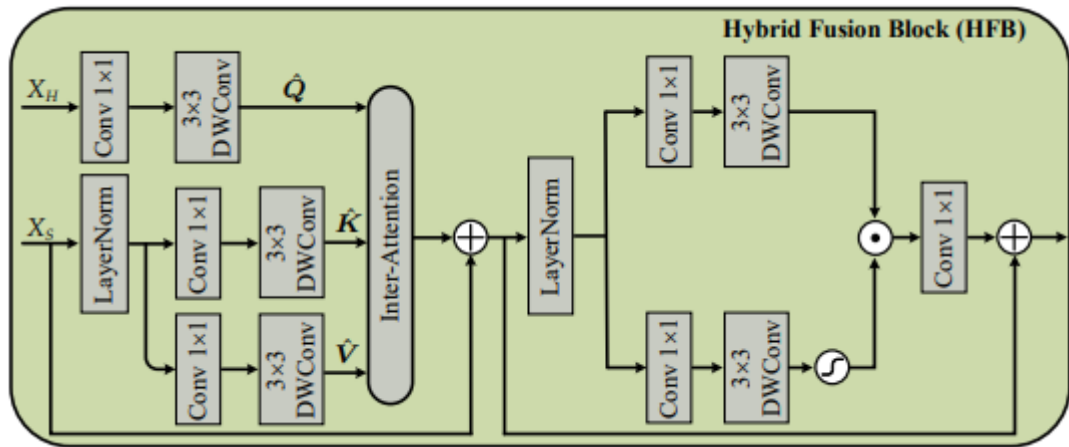
- To capture information from different axes, use two types of rectangle windows: horizontal and vertical windows.
- Split the attention heads into two equal groups and compute the self-attention within each group separately

SRWAB



HFB: Hybrid Fusion Block

- To better integrate the merits of CNN and transformer (HFERB and SRWAB), a hybrid fusion block (HFB) is used.
- Formulate the output of HFERB as the high-frequency prior query and the output of SRWAB as key, value and calculate the inter-attention to refine the global features which are obtained from SRWAB.
- To overcome loss of information through the channel an inter-attention based on the channel dimension to explore channel dependencies is used.
- A 1×1 convolutional layer followed by a 3×3 depth-wise convolutional layer to generate the high-frequency query $Q \in \mathbb{R}^{H \times W \times C}$ based on the output of HFERB
- As to the output of SRWAB normalize the features by the LayerNorm layer and then use the same operation as the query Q to get the key $K \in \mathbb{R}^{H \times W \times C}$ and the value $V \in \mathbb{R}^{H \times W \times C}$.
- Inter-Attention as $\text{Attention}(\hat{Q}, \hat{K}, \hat{V}) = \text{Softmax}\left(\frac{\hat{Q}\hat{K}^T}{\alpha}\right) \hat{V}$



Novelty Considerations

- **Adaptive SRWAB:** Prediction of window size for each small patch and attention mechanism after matching feature maps using interpolation (bilinear or trilinear)
- **Adaptive SRWAB:** Dynamic Window Size Selection for the entire image using softmax probability (predictive architecture before SRWAB)
- **Edge Aware CNN Modules** - Laplacian and MSD Convolution
- **Progressive Refinement Across Layers** using transformer blocks (capture the image and then recreate the image feature map) to create all low freq features first and then use the final low freq map for Cross-Attention fusion (Use Low freq as **Q** and CNN output as **K**)
- **Channel Attention for HFERB**
- **PTQ** strategy changes.

Novel Improvements on CRAFT

- **Adaptive SRWAB**
 - Dynamic Window Size Selection
- **(HFER Block) Edge Aware CNN Module**
 - Edge aware convolution + Laplacian Filters
 - Multi-Scale Dilated Convolution (of dilation = 1, 2, and 3)
- **(HF Block)** Refined and efficient fusion of features
 - Addition of Spatial Attention after basic fusion of features.

- The CRAFT model leverages the Shift Rectangle Window Attention Block (SRWAB) to perform window-based attention, balancing computational cost and performance.
- Originally, SRWAB used fixed window sizes, limiting its adaptability to diverse image content. The introduction of an adaptive SRWAB with learnable window sizes addresses this limitation, enabling dynamic selection of window sizes to optimize feature extraction.

For this we countered challenges like:

- Dynamic window size selection using a convolutional predictor.
- Robust tensor shape handling to prevent dimension mismatches.
- Integration with the Locality Complementary Module (LCM) for enhanced feature fusion.

Adaptive SRWAB: Dynamic Window Size Selection

The adaptive SRWAB enhances the original module by introducing a learnable window size mechanism and resolving implementation errors. Below are the detailed improvements.

To adapt to different image regions, a convolutional window size predictor was added to select optimal window sizes from a predefined set (e.g., $\{[2, 4], [4, 8], [8, 16]\}$).

Predictor Architecture: A sequential convolutional network:

Adp. Avg Pool $\rightarrow$ Conv2d($c, 64, 3, \text{stride} = 1, \text{padding} = 1$) $\rightarrow$ ReLU
 $\rightarrow$ Conv2d($64, |\mathcal{S}|, 1$)
 $\rightarrow$ Softmax($\text{dim} = 1$)

where c is the input channel dimension, and $|\mathcal{S}|$ is the number of window sizes (e.g., 3). The output is a probability map of shape $[b, |\mathcal{S}|, h, w]$, indicating the likelihood of each window size per spatial location.

Adaptive SRWAB: Dynamic Window Size Selection

- **Selection Mechanism:** The most likely window size index is first computed per spatial location using:

$$\text{predicted_classes} = \arg \max_{\text{dim}=1} (\text{window_probs}) \quad \text{of shape } [b, h, w]$$

These are then flattened per sample and passed to a mode operation:

$$\text{mode_size_idx} = \text{mode}(\text{predicted_classes.view}(b, -1), \text{dim} = 1) [0]$$

The final selected window size per sample is given by:

$$\text{selected_size} = \mathcal{S}[\text{mode_size_idx}]$$

- **Attention Modules:** Multiple `Attention_regular` modules are instantiated for each window size in $\mathcal{S}$, stored in `attns_h` and `attns_v`. For each sample in the batch, the attention module corresponding to its `mode_size_idx` is selected for horizontal and vertical attention computation.

Edge-Aware Convolution with Laplacian Filters

- **Motivation:**

- Edges are primary components of high-frequency details in SISR.
- Max-pooling is too generic, missing subtle edge patterns critical for sharp reconstructions.

- **Implementation:**

- Replaced max-pooling with a 3x3 convolutional layer initialized with Laplacian filters:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

- Learnable weights allow adaptation to diverse edge and texture patterns during training.
- Followed by 1x1 convolution for feature refinement.

Laplacian Filters: Technical Details

- **Laplacian Operator:** A second-order derivative filter that highlights regions of rapid intensity change (edges):

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Discrete form approximated by the 3x3 kernel above.

- **Advantages:**
 - Directly targets edge-like features, enhancing sharpness in reconstructed images.
 - Learnable convolution adapts to dataset-specific high-frequency patterns.
 - Preserves spatial resolution, unlike max-pooling.
- **Integration:** Applied to `mid_dim` channels in high-frequency branch, maintaining compatibility with HFERB's structure.

Multi-Scale Dilated Convolutions

Dilated convolutions Expand the receptive field of a convolution by adding gaps between kernel elements.

For example

A standard 3×3 convolution has a 3×3 receptive field. With a dilation rate of 2, the same 3×3 kernel covers a 5×5 area by skipping every other pixel. Multi-scale dilated convolutions here uses several dilated convolutions with different dilation rates in parallel. This allows the model to capture features at multiple scales simultaneously

We used different dilations to mimic different receptive fields:

- Dilation=1: Fine, local details.
- Dilation=2: Medium-scale patterns.
- Dilation=3: Broader contextual features.

Multi-Scale Dilated Convolutions: For the HFERB

- Modification: The high-frequency branch now includes **three parallel 3x3 dilated convolutions** with dilation rates of 1, 2, and 3.
- Combination: The **outputs** of these convolutions are **summed** to form a single multi-scale feature map, which is then refined with a 1x1 convolution.
- Integration: This **replaces** the original max-pooling and 1x1 convolution sequence.

Note: It was observed that using MSD Convolutions and Laplacian filters together didn't change the output much. One of the improvements will hence be selected based on experimentation and parameters.

Refined Fusion of Features: Addition of Spatial Attention

Original HFB Block

- Normalisation and Cross-Attention
- Fuse "low" and "high" freq data using the cross-attention mechanism
- Simple FFN consisting of convolutional layers, to refine the representation further.
- FFN output is added back to the fused features to yield the Output

New HFB Block

- Normalisation and Cross-Attention
- Fuse "low" and "high" freq data using the cross-attention mechanism
- We use **Spatial Attention** mechanism to fused features based on spatial importance.
- FFN as final layer (residual connection layer)

Spatial Attention

The **Spatial Attention module** is a lightweight addition that enhances the fused features by emphasizing spatially important regions.

- The fused features from the cross-attention step, with shape (B, dim, H, W) , where B is the batch size, dim is the channel dimension, and H and W are the height and width (**Input**)
- Use simple **1x1 Convolution** Reducing the channel dimension from dim to 1, producing a spatial attention map of shape $(B, 1, H, W)$. Each value in this map represents the importance of the corresponding spatial location.
- The attention map is passed through a **sigmoid function**, scaling its values between 0 and 1. This creates a soft weighting scheme.
- **Element-Wise Multiplication:** The attention map is multiplied element-wise with the fused features, modulating them spatially. Regions with higher attention values are amplified, while less important areas are suppressed.


```
#undef
```

```
psnr: 24.2129    Best: 24.2129 @ 502 iter
```

```
ssim: 0.6824    Best: 0.6824 @ 502 iter
```

```
#10 laplacian filter
```

```
psnr: 25.5507    Best: 25.5507 @ 502 iter
```

```
ssim: 0.7104    Best: 0.7104 @ 502 iter
```

```
#11 Adaptive SRWAB
```

```
psnr: 25.9903    Best: 25.9903 @ 502 iter
```

```
ssim: 0.7140    Best: 0.7140 @ 502 iter
```

*#11: Adaptive SRWAB + Multi-Scale Dilation Conv + Spatial Attention

Frequency-Guided PTQ Strategy

- Transformer-based SISR models are computationally expensive.
- Quantization reduces storage and inference cost by lowering precision (e.g., 8-bit, 4-bit) for weights and activations.
- Standard Quantization-Aware Training (QAT) requires extensive retraining, which is infeasible for many practical scenarios.
- PTQ allows post-training quantization without retraining, making it more efficient.
- * PTQ directly to transformers causes severe performance degradation due to high dynamic range in activations (difficult to quantize).
- * Loss of high-frequency details, essential for super-resolution.

Post-Training Quantization (PTQ)

Purpose: Dynamically optimizes clipping range for activations.

- Computes min/max values for activations.
- Uses histogram-based refinement to determine optimal clipping bounds.
- Optimizes clipping range iteratively to minimize quantization loss.
- b is the no. of bits to be used for quantization.

Mathematical Formulation:

$$\text{scale} = \frac{u - l}{2^b - 1}, \quad \text{zero point} = \text{clip} \left(\left\lfloor \frac{-l}{\text{scale}} \right\rfloor, 0, 2^b - 1 \right) \quad (1)$$

Quantization Process:

$$X_i^{\text{int}} = \text{clip} \left(\left\lfloor \frac{X_i}{\text{scale}} \right\rfloor + \text{zero point}, 0, 2^b - 1 \right) \quad (2)$$

- Converts floating-point input X_i into an integer representation X_i^{int} .
- Uses a scaling factor and zero point for precise mapping.

Adaptive Dual Clipping (ADC)

Purpose: Adaptive Dual Clipping (ADC) is a quantization technique designed to optimize the clipping bounds $\{l, u\}$ for minimizing quantization error. By adaptively adjusting these bounds, it improves the accuracy of low-bit quantization.

Methodology:

- Initialize clipping bounds l and u from the minimum and maximum values of the quantization layer F .
- Compute frequency-guided criteria using the *Frequency-Guided Criteria Measuring Processing (FCMP)* function.
- Iteratively refine l and u based on FCMP outputs, adjusting them dynamically to minimize information loss.
- Terminate when the optimal clipping bounds are found, ensuring minimal degradation in quantized model accuracy.

Boundary Refinement (BR)

Purpose: Optimizes clipping parameters for stable quantization using backpropagation.

- Refines learned bounds to minimize reconstruction loss.
- Uses straight-through estimation (STE) to fine-tune boundaries.

Loss Function:

$$L_{\text{rec}} = \frac{1}{B} \sum_{i=1}^B \|X_i - \hat{X}_i\|_1 \quad (3)$$

where X_i is the original activation, $\hat{X}_i$ is the quantized activation, and B is the batch size.

Frequency-Guided Optimization (FGO)

Purpose: Ensures high-frequency details remain intact post-quantization.

- Computes Fourier Transform (FFT) before and after quantization.
- Minimizes frequency-aware error to preserve details.
- This helps analyze how much high-frequency information is lost due to quantization.

Mathematical Formulation:

$$\gamma = \frac{1}{C} \sum_{i=1}^C |||F_X| - |F_{\hat{X}}||| \quad (4)$$

where F_X is the FFT of the original activation, and $F_{\hat{X}}$ is the FFT of the quantized activation.

Appendix: Scale and Zero Point Calculation

Converting Floating-Point to Integer:

$$\text{scale} = \frac{u - l}{2^b - 1} \quad (5)$$

$$\text{zero point} = \text{clip} \left(\left\lfloor \frac{-l}{\text{scale}} \right\rfloor, 0, 2^b - 1 \right) \quad (6)$$

- l, u are the minimum and maximum bounds of the input.
- Scale maps floating-point values into an integer range.
- Zero point ensures correct alignment.

Recovering Floating-Point Values:

$$\hat{X}_i = \text{scale} \cdot (X_i^{\text{int}} - \text{zero point}) \quad (7)$$

- Converts quantized integers X_i^{int} back to an approximate floating-point representation.

Ensures Values Stay Within Range:

$$\text{clip}(x; a, c) = \begin{cases} a, & \text{if } x < a \\ x, & \text{if } a \leq x \leq c \\ c, & \text{if } x > c \end{cases} \quad (8)$$

- Limits values between a and c to avoid out-of-range errors.

The End